

1.4.1 Loud Data Integrity Issues

Tools that process structured data are usually built with very specific expectations about what that data will look like. When the input matches those expectations, the tool works reliably. When it doesn't, the results can range from obvious errors to confusing failures that aren't immediately easy to diagnose.

This exercise focuses on **loud data integrity issues** — problems that cause an immediate and visible failure. In this case, you'll run the same alert analysis tool against two different JSON files. One file matches the expected format and will process successfully. The other contains an unexpected change that violates basic formatting rules and causes the tool to fail outright. By examining both files side by side, you'll identify exactly what changed and see how even small deviations from an expected data format can prevent tools from running at all.

Step 1: From the desktop, open **Firefox** and **OpenWebUI**.

Step 2: Download the files for this lab from the GitHub repository folder: **Labfiles1.4** and **save it to home/kali/Labfiles1.4**

Step 3: From OpenWebUI, start a new chat with **Gemini**.

Step 4: Upload **alert_file1.json** from **/home/kali/Labfiles1.4** directly into the chat, then enter the following prompt and note the response:

```
Prompt 1: Analyze the uploaded alert file and provide a structured summary.
```

Insights: This alert file follows the expected JSON structure and formatting rules, allowing gemini to parse it successfully. When input data matches expectations, the analysis runs as intended and produces a structured summary without errors.

Step 5: Start a new chat, upload **alert_file1 (copy).json** from **/home/kali/Labfiles1.4** into the chat, then enter the following prompt and note the response:

```
Prompt 2: Analyze the uploaded alert file and provide a structured summary.
```

Insights: This alert fails to load because the JSON is no longer valid. A syntax error prevents the parser from reading the file, causing the tool to stop immediately. This is a loud integrity failure — the problem is obvious and blocks analysis entirely.

Step 6: From the left menu, open the **terminal**.

Step 7: In the terminal, run the following commands:

```
cd /home/kali/Labfiles1.4
```

```
sha256sum alert_file1.json "alert_file1 (copy).json"
```

Insights: Even though one alert file is a copy of the other, a different hash confirms that something inside the file has changed. Since these alert files are automatically generated by a tool, their contents should not be modified manually. Hashing isn't useful for identifying what changed, but it is a reliable way to confirm whether a change occurred.

Step 8: From the desktop, open the **Labfiles1.4** folder and then open **alert_file1 (copy).json**.

Insights: Right away you can see the value that's causing the problem. The `tlp:white` value is supposed to be a string attribute but it's being highlighted by the editor because it's not enclosed in double quotes. In JSON, string values must always be quoted, and without them the file no longer conforms to the format.

Step 9: In the **alert_file1 (copy).json** file, enclose **tlp:white** in double quotes.

Note: The file may hang briefly when adding each quote.

Step 10: **Save and close the file.**

Step 11: In the terminal, run the hash command again:

```
cd /home/kali/Labfiles1.4
```

```
sha256sum alert_file1.json "alert_file1 (copy).json"
```

Insights: The hash values now match, confirming that the alert file has been fully restored to its original contents.

You've successfully explored loud data integrity issues.

1.4.2 Silent Data Integrity Issues

Unlike the previous exercise, not all data integrity issues cause obvious failures. Sometimes data can be modified in ways that still produce valid JSON and allow tools to run normally, even though the underlying meaning of the data has changed. These cases are especially dangerous because nothing breaks and there are no immediate signs that anything is wrong.

When this kind of data is later used to inform an AI model, the risk increases. The tool will pass along the altered values as if they were trustworthy, and the model will treat them as ground truth. In this exercise, you'll work with an alert file that has been subtly modified in a way that does not trigger an error in the tool. You'll use hashing as a routine integrity check to detect that the file has changed, then inspect the contents more closely to identify what was altered and how it could influence downstream AI analysis.

Step 1: Switch to **OpenWebUI** and start a new chat with **Gemini**.

Step 2: Upload **alert_file2.json** from **/home/kali/Labfiles1.4** directly into the chat, then enter the following prompt and note the response:

```
Prompt 1: Analyze the uploaded alert file and provide a structured summary.
```

Insights: This alert processes successfully because the JSON is still valid, but the silent changes significantly affect how the AI interprets the data. Lowering the threat level ID to 0 causes the model to downplay the risk, while changing the filename influences how the activity is framed and investigated. Because nothing fails, these kinds of changes can easily go unnoticed, making silent data integrity issues especially dangerous in AI-assisted analysis.

Step 3: From the desktop, open the **Labfiles1.4** folder and then open **alert_file2.json**.

Insights: When viewing the file directly, there are no obvious formatting errors or visual indicators that something is wrong. The JSON is valid and structured as expected, even though important values have been changed. This is what makes silent data integrity issues harder to detect — the data appears legitimate unless you already know what to look for.

Step 4: In the terminal, run the following commands:

```
cd /home/kali/Labfiles1.4
```

```
sha256sum alert_file1.json alert_file2.json
```

Insights: Comparing file hashes is especially effective in detecting silent data integrity issues. Since alert_file2.json is a copy of alert_file1.json, the hash mismatch quickly and easily confirms something changed.

Step 5: In the terminal, run the following commands:

```
python3 -m json.tool alert_file1.json > alert_file1.pretty.json
```

```
python3 -m json.tool alert_file2.json > alert_file2.pretty.json
```

Insights: JSON files are often stored in a compact, single-line format, which makes it difficult for comparison tools like diff to highlight individual changes. By "pretty-printing" the file first, the data is expanded into a line-by-line structure, allowing diff to compare specific fields and values instead of treating the entire file as one large difference.

Step 6: Next, run the **diff** tool to compare the pretty-printed files from the previous step:

```
diff -u alert_file1.pretty.json alert_file2.pretty.json
```

Insights: The diff command compares two files line by line and highlights exactly what changed between them.

Lines prefixed with "-" show values from the original file, while lines prefixed with "+" show values from the modified version. Diff lets you clearly see which specific fields were altered, making silent data integrity issues visible and easy to analyze.

Step 7: **Restore the file from a known-good backup.**

Insights: After identifying silent changes with hashing and diff, the safest response is often to restore the file from a known-good backup rather than attempting manual fixes.

Because these changes don't break tools or produce clear errors, there's no reliable point at which you can say the file is "correct" again. Multiple values may have been altered, and editing by hand risks introducing new, unintended changes. Restoring from backup is faster, more reliable, and ensures the data used by downstream tools and AI models can be trusted.

You've successfully explored silent data integrity issues.

Summary

This lab demonstrated two different classes of data integrity issues and how they affect tools and AI-assisted analysis in different ways.

	Loud Data Integrity Issues	Silent Data Integrity Issues
Indicators	• Tools fail immediately with errors, making the issue obvious.	• Tools and AI models process the data normally, trusting the modified values.
Why they're easier to detect / dangerous	• The failure provides a clear signal that something is wrong.	• There are no errors or warnings, and AI systems treat poisoned data as grounded truth.
Best defences	• Strict format validation • Schema checks • Automated parsing and early rejection • Restoring from a known-good copy if the file was modified	• Routine hash verification of trusted inputs • Comparing against known-good versions • Maintaining backups and restoring rather than manually fixing • Enforcing trust boundaries on data sources before AI ingestion